Cutting the cost of our AI coding assistant

Progress report for management · status at 10 October 2026 · Copilot workflow optimisation project

The goal. Our developers work with an AI coding assistant (GitHub Copilot CLI). It is billed in credits called **AIU**, and our spending has become large and uneven. This project aims to **cut that spending by about half (we expect 30–40%) while keeping the quality of the work the same.**

We do it in three steps: measure exactly where the credits go, remove the waste one lever at a time, and check every week, with numbers, that quality has not dropped.

≈140k

AIU spent in 203 sessions since cost records began on 24 July (a lower bound, see the chart note)

80%

of that spend is the cost of **carrying the conversation along**: re-reading and re-storing it at every step

71%

comes from only **31 long sessions**, which is 15% of all sessions with cost records

110 h

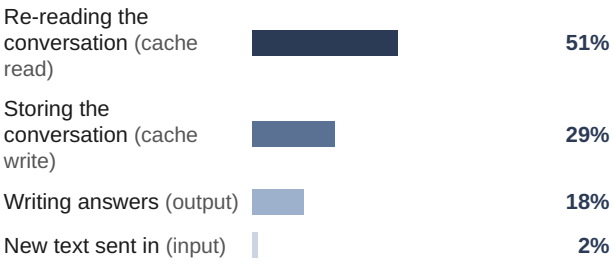
total time the assistant **waited for an approval click** over 7,436 requests, including time users were away

Why it costs so much, in plain words

Before every step, the assistant re-reads the whole conversation so far: on average about 180,000 tokens (≈135,000 words, longer than a typical novel). One re-read is cheap, but a single task takes dozens of steps, so carrying the conversation along adds up to 80% of the bill. Two things make it worse:

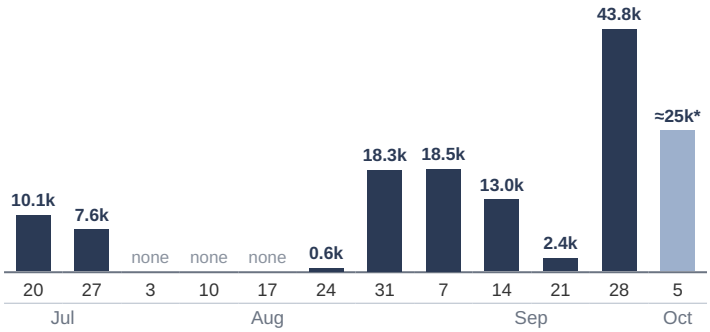
- **Pauses.** The assistant's short-term memory (the cache) expires after 5 minutes without activity. After a pause the whole history is paid for again, at about 12 times the price. These “cold restarts” are 19% of all spend: resuming a stale session costs ≈64 AIU for its first step, a fresh session only ≈11.
- **Autopilot.** In autopilot mode the assistant works alone for many steps in a row. The 91 autopilot sessions are 73% of the spend, and each costs 3.3 times more than a hands-on session.

Where the credits go (share of AIU)



The first two bars together are the 80%: the cost of carrying the conversation along.

Weekly spend, in thousands of AIU



Weeks start on Monday. No sessions were recorded 3–23 Aug. Cost records are incomplete for the weeks of 20 Jul, 27 Jul and 24 Aug (many sessions have none), so those bars understate the spend; from 31 Aug they are essentially complete. *Partial week (5–10 Oct); the normal-size conversation window applies from 9 Oct.

Bottom line

- **Done:** we can now see in detail where the credits go, and the biggest single lever, a normal-size conversation window, has been in force since 9 October (all 14 new sessions since then use it).
- **Not yet proven:** the saving itself. The first before/after comparison is possible around 16 October, after one week of data.
- **Needed from management:** nothing at this stage. This report is for information.

What we are doing about it

Each lever removes a different kind of waste. The percentages are upper-bound estimates taken from the project plan; they overlap, so they do not add up. Together we expect **30–40%**, with 50% as the target.

Lever	In plain words	Est. saving	Status
Normal-size conversation window by default	New sessions use the standard window (old history is summarised at about 160k tokens) instead of the very large one (about 750k). The large one stays one command away for the rare cases that need it.	15–20%	**Done** 9 Oct
Short fresh sessions with an automatic hand-over note	Instead of resuming a huge, stale session, start a new one that opens with a short summary (at most 1,500 tokens) of where things stand.	up to 8–10%	**Next to build**
Cheaper model for helper agents	The helpers that search and explore (10% of spend) use a cheaper model. The main assistant is unchanged.	3–4%	**From ≈16 Oct**
Better working habits	Standing instructions: combine steps, check on slow jobs less often, hand noisy build loops to a helper.	5–8%	**From ≈16 Oct**
Search past work instead of re-reading it	Look up old decisions and code on demand, so that conversations can stay short.	1–3%	**Planned**, only if a test supports it
Auto-approve safe read-only commands	Skip the “may I?” question for harmless commands such as `ls` or `git status` inside trusted folders.	≈0% (measured)	**Observing only**

What exists today

- **A private logbook of every assistant session** (a database on the workstation only): 393 sessions (203 of them with cost records), about 1,750 conversation turns, 35,900 tool calls, 20,600 model calls and 223,000 logged events, kept current by a background service. Secrets are masked before anything is stored.
- **A weekly report:** spend by model and session type, waiting time, quality indicators.
- **A permission assistant in observation mode** since 10 October. It records what it *would* approve and approves nothing. Each decision takes about 10–14 ms (the limit is 50 ms).
- **Settings changes** made on 9 October: normal-size window by default, a one-word switch to the large window, and a per-session spending cap.
- **Tested:** 319 automated tests pass; a 19-page plan records decisions, risks and estimates.

What we learned so far

- **The measurements can be trusted.** At its first run the logbook reproduced the earlier hand-made baseline almost exactly (129,366 AIU against ≈129.4k; the total has grown since). Row counts match their sources, re-running adds no duplicates, and none of the 2.25 million text fields checked contained a secret after masking.
- **One lever turned out smaller than hoped (a correction).** We expected the permission assistant to save 2–4% by removing long waits. Replayed on 4,536 real past command-approval requests, it would have answered 9% of them, and none of the 43 that people refused or cancelled. But those were already the quick ones: together they account for 0.1% of the waiting time and for none of the 118 waits of 5 minutes or more. We therefore do not count on it; the saving has to come from the other levers.
- **Spend is lumpy.** A handful of long sessions decide the bill (page 1), so the levers aim at shortening those, not at everyday use.

Quality safeguard

A saving only counts if quality holds. The logbook tracks automatic indicators; today's baseline is **3.0 mid-task stops by a user per 100 messages**, **1.3 errors or retries per 100 assistant turns** and **276** tasks marked complete. Any lever that makes an indicator more than 25% worse is rolled back.

Cost and expected benefit

Building and testing this system used ≈6.6k AIU in the project's own sessions, about 5% of all recorded spend (the assistant helped to build it). Take the average of ≈12.4k AIU per week, a cautious figure because it includes the three empty August weeks and the weeks with incomplete cost records (the five full weeks since 31 Aug averaged ≈19k). A 30–40% saving is then ≈3.7–5.0k AIU per week, so the build would be repaid in under two weeks once the levers are active. **This is a projection: no saving has been measured yet.**

Risks and open points

- **Projections, not results.** The estimates overlap; the weekly report will show what really happens.
- **Shorter sessions could lose context.** Mitigation: automatic hand-over notes and searchable history; the large window stays available on request.
- **Permission assistant.** It can only ever say “approved”, and only for a short list of read-only commands inside trusted folders. It never blocks or runs anything, and any error falls back to the normal question. It would be switched on only after a 3-day log review and an explicit human decision, and one setting turns it off again.
- **Data stays local.** The logbook and every project tool run on the workstation only; the database accepts connections from this machine alone, nothing is sent to an outside service, and secrets are masked before storage.
- **Open point:** the per-session spending cap (1,000 AIU) is missing in about half of the new sessions (6 of 11 real sessions since 9 Oct, test runs excluded). The cause is not yet known.

Next steps

When	What
13 Oct (earliest)	Review the 3-day observation log of the permission assistant and decide whether to switch it on. This is optional, because the payoff is small.
≈16 Oct	One week of data on the new defaults: first before/after comparison. Start the cheaper helper models and the working habits.
After that	Hand-over notes for short fresh sessions, then history search. Each step is checked against the quality indicators.
Every week	Review the weekly report and decide on further levers.

Decision needed from management: none for now. Feedback on the 50% target or on the quality indicators is welcome.

Glossary

AIU	The credit unit in which GitHub bills Copilot usage.
Token	A piece of text; about three quarters of a word.
Conversation window (context)	The history the assistant re-reads before every step.
Cache	Short-term memory that makes re-reading cheap. It expires after 5 minutes of inactivity.
Autopilot	The assistant works through a task on its own, for many steps in a row.
Helper agent	A second assistant that the main one hands a side task, such as searching the code.
Approval request	The assistant asking “may I run this command?” before it acts.

Sources: project database `copilot_ctx` (10 Oct 2026, about 11:30), the project plan (`PLAN.md`, §2, §6.12, §6.13, §10) and a replay of 4,536 historical approval requests. Figures are rounded and keep growing as sessions continue; spend totals count only sessions that have cost records, so they are lower bounds.